Module 1: Data Manipulation with dplyr

Lesson 3 | Prepare and summarize the textbook data

Table of contents

Purpose and data 1
Select columns 2
Filter rows 2
Pipes and readable steps 3
Arrange and create variables 3
Count categories and summarize groups 4
Missing values and duplicates 4
Join on a key 5
Reshape a summary 5
Export a derived result 6

Purpose and data

Data analysis starts with a clear question and a well-defined table. This lesson uses the checked Restaurant CSV that we will revisit in Module 2. We focus on R operations here; the statistical interpretation comes later.

An analyst wants to select restaurants by rating and price, then summarize prices within each quality group. **One row is one restaurant**, identified by `restaurant_id`. Prices are in USD, and ratings are ordered Good, Very Good, Excellent.

يريد محلّل اختيار المطاعم حسب تقييم الجودة والسعر، ثم تلخيص الأسعار داخل كل فئة من فئات الجودة. **يمثّل كل صف مطعمًا واحدًا** يحدّده المتغيّر `restaurant_id`. الأسعار بالدولار الأمريكي، وفئات الجودة مرتبة: جيد، وجيد جدًا، وممتاز.

```
# library(): loads a package so its functions can be used in this session.
# read.csv(): reads a comma-separated file into a data frame; its first row supplies
 ↪  column names by default.
# c(): combines values into a vector, converting them to a common type when needed.
# head(): shows the first n entries or rows, six by default.
# glimpse(): shows table dimensions, column types, and example values.
# factor() gives categories explicit levels and an optional order.
# The level order controls grouping and display; ordered=TRUE records ordinal category
 ↪  order.
library(dplyr)
restaurants <- read.csv("data/module02/restaurant_ratings.csv")
restaurants$quality_rating <- factor(restaurants$quality_rating,
  levels = c("Good", "Very Good", "Excellent"), ordered = TRUE)
head(restaurants)
```

```
  restaurant_id quality_rating meal_price_usd
1             1           Good             18
2             2      Very Good             22
3             3           Good             28
4             4      Excellent             38
5             5      Very Good             33
6             6           Good             28
```

```
glimpse(restaurants)
```

```
Rows: 300
Columns: 3
$ restaurant_id  <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
$ quality_rating <ord> Good, Very Good, Good, Excellent, Very Good, Good, Very~
$ meal_price_usd <int> 18, 22, 28, 38, 33, 28, 19, 11, 23, 13, 33, 44, 42, 34,~
```

Use the RStudio project so these paths resolve from the 20261 folder. Keep the source CSV unchanged. Assign transformed tables to new objects.

Select columns

`select()` chooses variables. Use meaningful names rather than column positions when practical.

```
# head(): shows the first n entries or rows, six by default.
# select() retains or renames the specified columns.
# It changes the variables included in the result without filtering observations.
prices <- restaurants |> select(restaurant_id, meal_price_usd)
head(prices)
```

```
  restaurant_id meal_price_usd
1             1             18
2             2             22
3             3             28
4             4             38
5             5             33
6             6             28
```

Filter rows

`filter()` retains observations meeting conditions. A comma combines conditions with AND. Use `|` for OR and `%in%` for membership in a set.

```
# head(): shows the first n entries or rows, six by default.
# nrow(): returns the number of rows in a table or matrix.
# c(): combines values into a vector, converting them to a common type when needed.
# filter() keeps rows whose conditions are TRUE; commas combine conditions with AND.
# Missing comparisons do not pass the filter; the source object stays unchanged unless
 ↪  reassigned.
selected <- restaurants |>
  filter(quality_rating == "Excellent", meal_price_usd >= 30)
head(selected)
```

```
  restaurant_id quality_rating meal_price_usd
1             4      Excellent             38
2            13      Excellent             42
3            14      Excellent             34
4            23      Excellent             32
5            24      Excellent             33
6            33      Excellent             44
```

```
nrow(selected)
```

```
[1] 50
```

```
restaurants |>
  filter(quality_rating %in% c("Good", "Very Good"), meal_price_usd < 20) |>
  head()
```

```
  restaurant_id quality_rating meal_price_usd
1             1           Good             18
2             7      Very Good             19
3             8      Very Good             11
4            10           Good             13
5            20           Good             19
6            36           Good             10
```

Missing values need deliberate treatment: a comparison that evaluates to NA will not pass `filter()`. Check for missingness before interpreting the remaining rows.

Pipes and readable steps

The base pipe |> sends one result into the next operation. Read the chain from top to bottom. `dplyr` also supports %>%; use one style consistently in a report.

```
# select(): chooses columns by name or selection rule.
# arrange(): orders rows by supplied columns; desc() reverses a column to descending
 ↪  order.
# desc(): reverses sorting order to descending inside arrange().
# head(): shows the first n entries or rows, six by default.
# filter() keeps rows whose conditions are TRUE; commas combine conditions with AND.
# Missing comparisons do not pass the filter; the source object stays unchanged unless
 ↪  reassigned.
restaurants |>
  filter(meal_price_usd >= 30) |>
  select(restaurant_id, quality_rating, meal_price_usd) |>
  arrange(desc(meal_price_usd), restaurant_id) |>
  head(5)
```

```
  restaurant_id quality_rating meal_price_usd
1            67      Very Good             48
2           170      Excellent             48
3           271      Excellent             48
4            35      Excellent             47
5           269      Excellent             47
```

Arrange and create variables

`arrange()` sorts rows. `desc()` reverses an ordering. `mutate()` creates or changes columns while retaining the rows.

```
# mutate(): creates or replaces columns while retaining the original rows.
# c(): combines values into a vector, converting them to a common type when needed.
# arrange(): orders rows by supplied columns; desc() reverses a column to descending
 ↪  order.
# head(): shows the first n entries or rows, six by default.
# cut() assigns numerical values to intervals defined by breaks.
# Check right and include.lowest to determine which interval contains a boundary value.
prepared <- restaurants |>
  mutate(price_at_least_30 = meal_price_usd >= 30,
         price_band = cut(meal_price_usd, c(10,20,30,40,50), right = FALSE)) |>
  arrange(quality_rating, meal_price_usd)
head(prepared)
```

```
  restaurant_id quality_rating meal_price_usd price_at_least_30 price_band
1            36           Good             10             FALSE    [10,20)
2            52           Good             10             FALSE    [10,20)
3            60           Good             10             FALSE    [10,20)
4           125           Good             10             FALSE    [10,20)
5           182           Good             10             FALSE    [10,20)
6           189           Good             10             FALSE    [10,20)
```

The price bands include the lower boundary and exclude the upper one. These are derived labels, not corrections to the original price.

Count categories and summarize groups

`count()` counts observations by categories. `group_by()` defines groups for subsequent operations. `summarise()` reduces each group to summary rows.

```
# count(): counts rows for each combination of supplied categories.
# n(): counts rows in the current dplyr group.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# median(): returns the middle sorted value, averaging the middle pair for even sample
 ↪  sizes.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# nrow(): returns the number of rows in a table or matrix.
# summarise() reduces each group to the statistics specified here.
# group_by() defines the groups; .groups = "drop" removes grouping from the result.
restaurants |> count(quality_rating, name = "restaurants")
```

```
  quality_rating restaurants
1           Good          84
2      Very Good         150
3      Excellent          66
```

```
group_summary <- restaurants |>
  group_by(quality_rating) |>
  summarise(n = n(), mean_price_usd = mean(meal_price_usd),
            median_price_usd = median(meal_price_usd), .groups = "drop")
group_summary
```

```
# A tibble: 3 x 4
  quality_rating     n mean_price_usd median_price_usd
  <ord>          <int>          <dbl>            <dbl>
1 Good              84           19.7             19.5
2 Very Good        150           25.6             25
3 Excellent         66           34.3             34.5
```

```
stopifnot(sum(group_summary$n) == nrow(restaurants))
```

Using `.groups = "drop"` makes the result ungrouped. Otherwise, grouping metadata can affect later calculations. `ungroup()` explicitly removes grouping when needed. Do not average arbitrary numeric category codes.

Missing values and duplicates

```
# colSums(): sums each column separately.
# is.na(): marks missing values with TRUE; na.rm=TRUE elsewhere removes missing values
 ↪  only when justified.
# anyDuplicated() returns 0 when no duplicate value is found, otherwise its first
 ↪  position.
# Check identifiers for duplicate records; equal measurements alone do not imply
 ↪  duplicates.
colSums(is.na(restaurants))
```

```
 restaurant_id quality_rating meal_price_usd
             0              0              0
```

```
anyDuplicated(restaurants$restaurant_id)
```

```
[1] 0
```

A duplicate identifier and two restaurants with the same price are different issues. Investigate before dropping rows. Use `na.rm = TRUE` only when you can explain which observations are omitted and how that affects the summary.

Join on a key

A join combines tables through shared keys. This lookup is **derived from our own group summary**. Each quality category occurs once in the lookup, so a left join should preserve the 300 restaurant rows.

```
# select(): chooses columns by name or selection rule.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# nrow(): returns the number of rows in a table or matrix.
# anyNA(): returns TRUE if any value is missing; !anyNA(...) requires no missing values.
# head(): shows the first n entries or rows, six by default.
# left_join() adds matching lookup information using the specified key.
# Repeated lookup keys can multiply rows, so check key uniqueness and the result row count.
lookup <- group_summary |>
  select(quality_rating, group_median_usd = median_price_usd)
with_group_median <- restaurants |>
  left_join(lookup, by = "quality_rating")
stopifnot(nrow(with_group_median) == nrow(restaurants),
          !anyNA(with_group_median$group_median_usd))
head(with_group_median)
```

```
  restaurant_id quality_rating meal_price_usd group_median_usd
1             1           Good             18             19.5
2             2      Very Good             22             25.0
3             3           Good             28             19.5
4             4      Excellent             38             34.5
5             5      Very Good             33             25.0
6             6           Good             28             19.5
```

Repeated keys on both sides can multiply rows. Check key uniqueness and row counts rather than assuming a join is harmless. `inner_join()` keeps matching keys; `left_join()` retains all rows from the left table.

Reshape a summary

`pivot_longer()` moves selected columns into name/value rows. `pivot_wider()` reverses that organization when the keys uniquely identify cells.

```
# library(): loads a package so its functions can be used in this session.
# select(): chooses columns by name or selection rule.
# pivot_wider(): turns name-value rows into columns using names_from and values_from.
# pivot_longer() turns selected columns into name-value rows.
# names_to and values_to name the new columns; untouched columns identify the original
 ↪  groups.
library(tidyr)
long_summary <- group_summary |>
  select(quality_rating, mean_price_usd, median_price_usd) |>
  pivot_longer(-quality_rating, names_to = "measure", values_to = "price_usd")
long_summary
```

```
# A tibble: 6 x 3
  quality_rating measure          price_usd
  <ord>          <chr>                <dbl>
1 Good           mean_price_usd        19.7
2 Good           median_price_usd      19.5
3 Very Good      mean_price_usd        25.6
4 Very Good      median_price_usd      25
5 Excellent      mean_price_usd        34.3
6 Excellent      median_price_usd      34.5
```

```
long_summary |>
  pivot_wider(names_from = measure, values_from = price_usd)
```

```
# A tibble: 3 x 3
  quality_rating mean_price_usd median_price_usd
  <ord>                   <dbl>            <dbl>
1 Good                     19.7             19.5
2 Very Good                25.6             25
3 Excellent                34.3             34.5
```

Export a derived result

Run this manually when you want a report output. Generated results belong in `_output/`, separate from input data.

```
# dir.create() prepares an output folder; write.csv() saves the derived summary as a CSV.
# row.names = FALSE omits row-number columns and leaves the original input file unchanged.
dir.create("_output", showWarnings = FALSE)
write.csv(group_summary, "_output/restaurant-price-summary.csv", row.names = FALSE)
```

Practice Lesson 3. The source CSV and verification limits are documented in the data guide.